Dissertation Title

AI-Driven Intelligent Test Automation Framework for End-to-End Quality Assurance

Course No.	S1-25_CCZG628T
Course Title	Dissertation
Dissertation Work Done by	Revathi C
Student Name	Revathi C
BITS ID	2024TM93022
Degree Program	M.Tech in Software Engineering
Research Area	A Retrieval-Augmented and LLM-Driven Intelligent Test Automation Framework for End-to-End Quality Assurance
Dissertation carried out at	DELL Technologies, Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

VIDYA VIHAR, PILANI, RAJASTHAN - 333031.

CONTENTS

INTRODUCTION 4
- Background 4
- Problem Statement 4
- Proposed Solution 4

OBJECTIVES 5
ABSTRACT 6
SYSTEM ARCHITECTURE 7
- Overview 7
- Agent Roles 7
- Multi-Agent Interaction 8
- Technology Stack 9

DETAILED SYSTEM DESIGN 10
- Intermediate Representation (IR) 10
- End-to-End Data Flow 11
- RAG Knowledge Base Design 11
- RAG Knowledge Base Update Strategy (New Release) 12
- Validation Pipeline 12
- Security: LLM API Risk Mitigation 13

IMPLEMENTATION PROGRESS 15
- Development Environment 15
- Implemented Components 15
- Validated Test Execution Results 16
- Module Progress 16

WORK COMPLETED 17
SCOPE OF WORK AND LIMITATIONS 19
PLAN OF WORK 20
CONCLUSION 21
ABBREVIATIONS 22
LITERATURE REFERENCES 23

List of Figures

Fig 1 – Overall System Architecture 7
Fig 2 – Multi-Agent Interaction & Shared State 8
Fig 3 – End-to-End Data Flow Diagram (with Feedback Loop) 8
Fig 4 – Intermediate Representation (IR) Structure 9
Fig 5 – RAG Pipeline – Ingestion, Retrieval & LLM Generation 9
Fig 6 – Validation & Human-in-the-Loop Review Pipeline 10
Fig 7 – Module-wise Implementation Progress 12

1. Introduction

1.1 Background

Enterprise software development requires rigorous validation across numerous features, API endpoints, and functional scenarios. At organisations like Dell Technologies, test cases for complex systems such as OpenManage Enterprise (OME) are authored in dedicated test management platforms, while the knowledge required to automate them is scattered across High-Level Design (HLD) documents, Business Specification (BSpec) files, API guides, and existing automation repositories. This fragmented landscape forces automation engineers to manually interpret natural-language test steps, cross-reference multiple documents, and hand-craft test scripts — a process that is slow, error-prone, and difficult to scale as product complexity grows.

The growing adoption of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) in software engineering opens new possibilities for intelligent test automation. By grounding code generation in retrieved system artefacts and verified repository methods, LLMs can produce automation scripts that are contextually accurate, reusable, and maintainable — while deterministic guardrails prevent hallucinations and enforce quality standards.

1.2 Problem Statement

The key challenges addressed by this project are:

- Disconnection between test management tools and automation artefacts (HLD, BSpec, API specs)
- Manual interpretation of natural-language test steps into executable automation code
- Low code reuse across test scripts due to absence of structured repository indexing
- Poor traceability from test logic back to business requirements and API specifications
- High maintenance overhead when product APIs evolve and test scripts must be updated
- Difficult root-cause analysis when automated tests fail without specification linkage
- Repeated manual effort identifying reusable API clients, builders, and validation utilities

1.3 Proposed Solution

This dissertation proposes an AI-Driven Intelligent Test Automation Framework that applies NLP, RAG, and a constraint-driven multi-agent architecture to convert natural-language API test cases into reliable, maintainable automation scripts. The framework ingests test cases from qTest Manager via REST API, converts steps into a structured Intermediate Representation (IR), indexes HLD, BSpec, API specifications, and existing automation code into a vector-based RAG store, and uses a Planner–Retriever–Coder–Validator multi-agent pipeline to generate and validate scripts.

Generated scripts undergo a multi-stage automated validation pipeline and mandatory human review via pull requests before execution on live systems. Execution logs, API responses, and reviewer feedback are fed back into the retrieval and mapping logic, enabling continuous improvement of output quality over time. This constraint-driven design ensures correctness and maintainability without relying solely on the probabilistic outputs of LLMs.

2. Objectives

The following nine objectives define the full scope of this dissertation:

1. **Test Management Tool Integration:** Fetch test cases, test steps, preconditions, and test data from qTest Manager through its REST API.

2. **Step Understanding and Structuring:** Convert natural-language test steps into a structured Intermediate Representation (IR) capturing actions, objects, inputs, and expected results using NLP techniques.

3. **RAG Knowledge Base Construction:** Ingest and index HLD documents, BSpec files, API guides, UI/API contracts, and automation standards into a vector-based searchable knowledge store.

4. **Repository Selection:** Identify the appropriate target automation repository based on application module, framework type, and coding standards.

5. **Library and Method Discovery:** Retrieve reusable functions, API clients, page objects, and utilities from existing repositories using semantic similarity between step IR and method metadata.

6. **Test Script Generation:** Generate end-to-end executable automation scripts using LLMs guided by retrieved context, with guardrails enforcing approved dependencies, templates, and lint rules.

7. **Validation Pipeline:** Ensure generated scripts pass compilation, linting, OpenAPI schema compliance, authentication verification, and a basic dry-run where applicable.

8. **Human-in-the-Loop Review:** Produce pull requests and capture reviewer feedback to iteratively improve retrieval and IR mapping logic over time.

9. **Outcome Measurement:** Evaluate reduction in automation creation time, increase in code reuse, and improvement in script stability and maintainability.

3. ABSTRACT

This dissertation proposes an AI-Driven Intelligent Test Automation Framework that automatically converts natural-language REST API test cases from qTest Manager into executable, validated Python pytest scripts for Dell OpenManage Enterprise (OME). The framework bridges the critical gap between test management tools and scattered system artefacts (HLD documents, BSpec files, API specifications, automation repositories).

The core pipeline employs four specialised agents — Planner, Retriever, Coder, and Validator — coordinating through a shared Intermediate Representation (IR). A RAG knowledge base indexes system artefacts for semantic retrieval, grounding LLM code generation in authoritative context. Multi-stage guardrails (repository-only dependencies, OpenAPI schema validation, lint checks, human-in-the-loop PR review) ensure correctness.

The framework has been validated with two real Dell OME test cases on a live system: TC-174995 (11 steps, 1.63 s, PASSED) and TC-180015 (8 steps, 192 s, PASSED). Template-based generation is complete; LLM-guided generation is 80% complete. The system demonstrates significant potential to reduce manual automation effort and accelerate test development cycles for enterprise REST API platforms.

The framework's design prioritises maintainability and scalability through constraint-driven coordination. Rather than relying solely on LLM probabilistic outputs, the system enforces deterministic guardrails at every stage: the Planner produces bounded execution plans; the Retriever grounds generation in indexed system artefacts; the Coder uses approved templates and repository-only imports; and the Validator applies rule-based checks independent of LLM confidence scores. This multi-layered approach significantly reduces hallucinations and ensures generated scripts are correct, maintainable, and aligned with.

Signature of Student	**Signature of Supervisor**	**Signature of Additional Examiner**
Name: Revathi C	**Name:** Nithya V.V	**Name:** Chandiramouli Ravisankar

4. System Architecture

4.1 Overview

The AI-Driven Test Automation Framework is structured as a sequential, multi-agent pipeline (Figure 1). Test cases fetched from qTest Manager are parsed into an Intermediate Representation that drives four specialised agents: Planner, Retriever, Coder, and Validator. Agents coordinate through a shared structured state rather than direct message passing, ensuring clean separation of concerns. Generated scripts undergo a multi-stage validation pipeline and mandatory human review before execution on live systems. Execution feedback loops back into the retrieval and mapping components to continuously improve output quality.

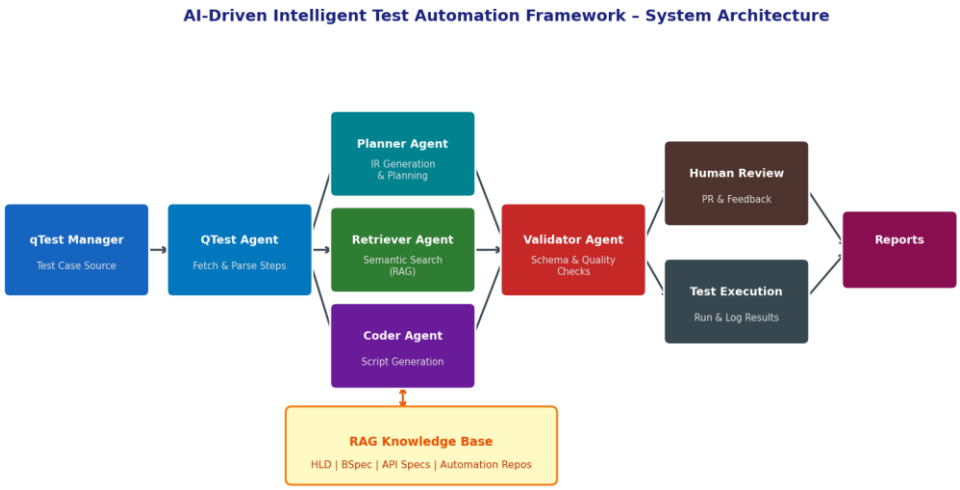


Fig 1: Overall System Architecture of the AI-Driven Test Automation Framework

4.2 Agent Roles

Agent / Component	Role
QTest Agent	Connects to qTest Manager REST API to fetch test case details, steps, and preconditions. Outputs raw step text for downstream NLP processing.
Planner Agent	Parses natural-language steps into a structured IR. Derives the ordered sequence of API calls, identifies inter-step dependencies, and produces a constrained execution plan that guides all subsequent agents.
Retriever Agent	Performs semantic similarity search over the RAG Knowledge Base using IR fields as queries. Returns the top-K most relevant HLD sections, BSpec clauses,

	API endpoint schemas, and reusable repository methods.
Coder Agent	Maps the IR to retrieved context and generates executable Python pytest scripts. Enforces repository-only dependencies to prevent unsupported imports or hallucinated method calls. Supports template and LLM modes.
Validator Agent	Enforces correctness through OpenAPI schema validation, authentication checks, lint rules, and dry-run execution. Triggers Coder Agent regeneration on failure with error context appended to the prompt.
Orchestrator	Coordinates the full pipeline, manages retry loop on validation failure, triggers human review PRs, feeds back reviewer comments, and produces JSON + HTML execution reports with per-step logs and timing.

4.3 Multi-Agent Interaction

Figure 2 illustrates how the four core agents interact through a shared structured state rather than direct peer-to-peer messaging. The Planner populates the state with an execution plan; the Retriever appends retrieved artefacts; the Coder reads both to generate the script; and the Validator updates the state with a pass/fail verdict and failure context. This pattern ensures each agent operates on consistent, versioned data and prevents race conditions in coordination.

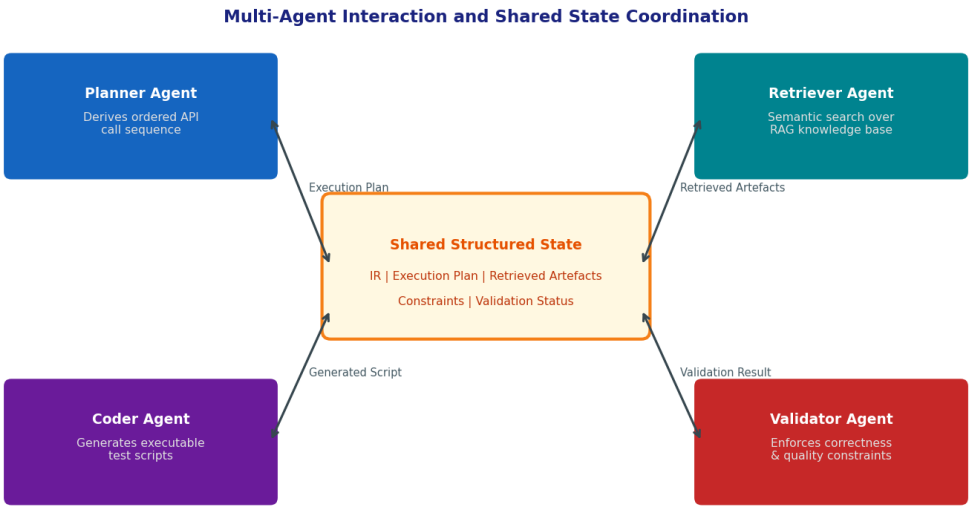


Fig 2: Multi-Agent Interaction Diagram – Shared Structured State Coordination

4.4 Technology Stack

Component	Technology / Choice
Language	Python 3.10+
Test Framework	pytest with OmeBaseFunctionalTestCase lifecycle (atlas-ome-staging)
NLP / IR Parsing	Regex-based step classifier with semantic intent mapping
RAG / Vector Store	FAISS / ChromaDB for embedding-based retrieval
LLM Integration	OpenAI GPT-4 / Azure OpenAI (optional; templates used by default)
API Client	requests library; atlas-ome-staging framework (Dell internal)
Test Management	qTest Manager REST API (read-only, Bearer-token auth)
Validation	OpenAPI/Swagger schema validation, flake8 linting, dry-run
Reporting	JSON + HTML reports with per-step logs and timing
Version Control	Git / GitHub for human-in-the-loop PR reviews

5. Detailed System Design

5.1 Intermediate Representation (IR)

The IR is the central data structure bridging natural-language test steps and agent operations. Each test step is parsed into seven structured fields that act as a shared contract across all agents, as shown in Figure 4.

IR Field	Description
action	Verb identifying the operation (GET, POST, VERIFY, DISCOVER, CONFIGURE)
object	Target resource or API endpoint (e.g., /api/DeviceService/Devices)
inputs	Request parameters, payloads, and authentication credentials
expected_result	Assertion condition (e.g., status_code == 200, field value match)
http_method	Inferred HTTP verb (GET, POST, PUT, DELETE, PATCH)
auth_type	Authentication mechanism (Bearer token, Basic auth, API key)
dependencies	Step IDs that must complete successfully before this step executes

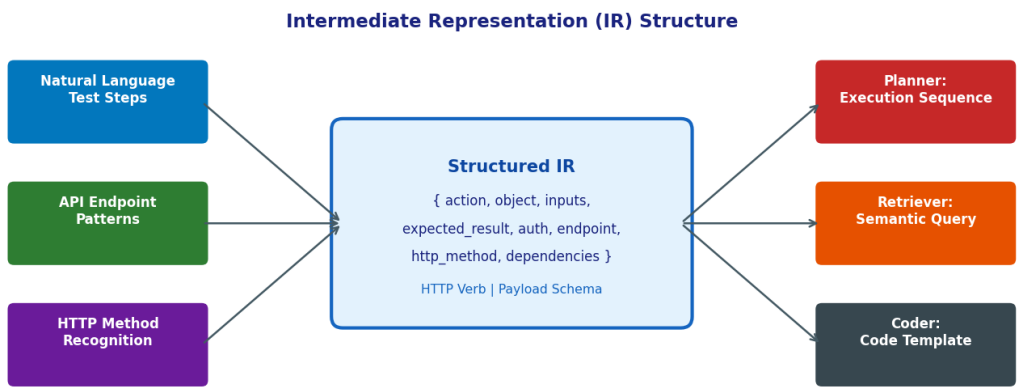


Fig 4: Intermediate Representation (IR) – Input Parsing to Agent Consumption

5.2 End-to-End Data Flow

Figure 3 shows the complete data flow from test case input through NLP parsing, RAG retrieval, script generation, validation, human review, and test execution. A continuous feedback loop connects the human review stage back to both the NLP parsing and RAG retrieval stages, enabling reviewer corrections to improve future generation accuracy over time.

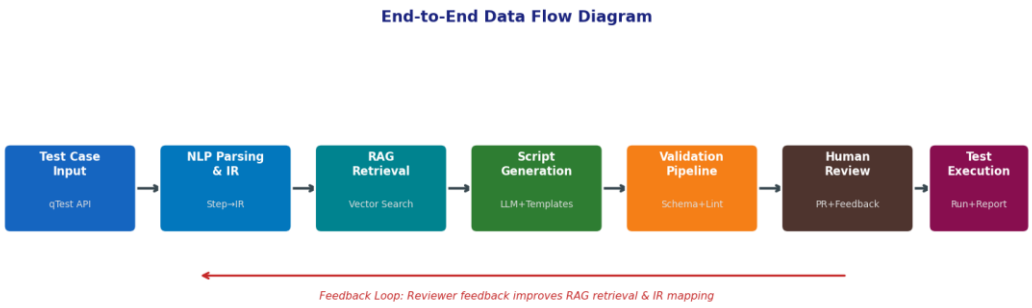


Fig 3: End-to-End Data Flow from Test Case Input to Execution Report

5.3 RAG Knowledge Base Design

Four categories of documents are ingested into the RAG Knowledge Base (Figure 5): HLD Documents describing functional behaviour and data flows; BSpec Files defining business rules and acceptance criteria; API Specifications (OpenAPI/Swagger) providing endpoint schemas, request/response formats, and auth requirements; and Automation Repositories indexed for reusable API clients, request builders, and validation utilities.

Documents are chunked at section boundaries, embedded using a sentence-transformer model, and stored in a FAISS or ChromaDB vector index. At inference time, the Retriever Agent constructs a composite query from IR fields (action + object + expected_result) and retrieves the top-K most semantically similar chunks to provide grounded context for the Coder Agent.

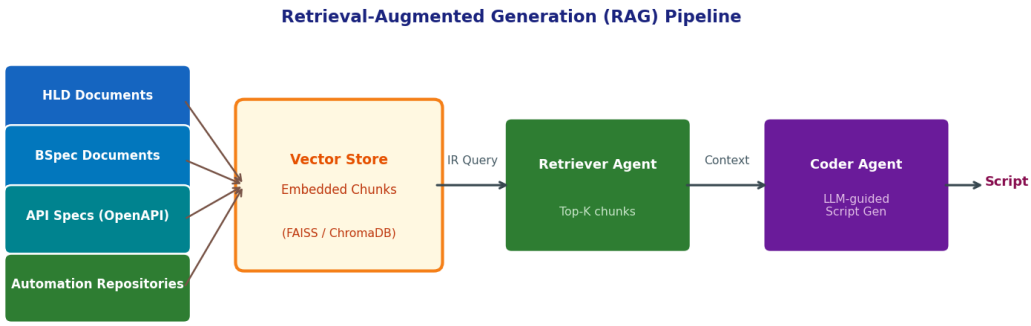


Fig 5: RAG Pipeline – Knowledge Base Ingestion, Semantic Retrieval, and LLM-Guided Generation

5.4 RAG Knowledge Base Update Strategy (New Release)

When a new product release is planned, the RAG knowledge base is updated through a structured incremental update pipeline to ensure the Retriever always operates on current artefacts. The update strategy involves the following steps:

Update Mechanism	Description
Document Versioning	Each artefact (HLD, BSpec, API specification) is tagged with the release version as metadata at ingestion time. This allows version-gated retrieval — the Retriever Agent can be pinned to a specific release to prevent mixing artefacts across versions.
Hash-Based Change Detection	Each document chunk is fingerprinted with a content hash. On a new release, only modified or newly added chunks are re-embedded and re-indexed into FAISS/ChromaDB, significantly reducing re-indexing time compared to a full rebuild.
Full Re-Index Trigger	A complete re-index is triggered when major structural changes occur — such as a new API endpoint category being introduced or an existing BSpec being fully rewritten. This is detected via a diff threshold on the number of changed chunks.
CI/CD Pipeline Integration	The RAG update pipeline is invoked as an automated step in the release CI/CD pipeline, executed immediately after the documentation freeze for each release. This ensures the knowledge base is always aligned with the latest released artefacts.
Stale Chunk Retirement	Chunks not updated across N consecutive release cycles are automatically flagged and queued for review. Obsolete chunks are removed to prevent outdated context from influencing code generation in future releases.
Scheduled Re-Index	A scheduled weekly re-index job runs independently of release events to catch documentation drift, interim patch notes, and hotfix documentation updates.

5.5 Validation Pipeline

Generated scripts pass through six validation stages before reaching the human reviewer (Figure 6). Scripts that fail any automated check are rejected and sent back to the Coder Agent for regeneration with the failure context appended to the generation prompt.

Stage 1 – OpenAPI Schema Check: All API calls validated against ingested OpenAPI spec for correct paths, methods, request schemas

Stage 2 – Auth Verification: Confirms correct auth mechanism and credential placeholders are present

Stage 3 – Lint and Formatting: Applies flake8 and autopep8 to enforce coding standards

Stage 4 – Dry-Run / Build Check: Imports and syntax-checks the module in isolation without live execution

Stage 5 – Consistency Check: Verifies all IR steps have a corresponding code block in the generated script

Stage 6 – Human Review (PR): Reviewer approves/rejects via pull request; feedback stored to improve RAG retrieval

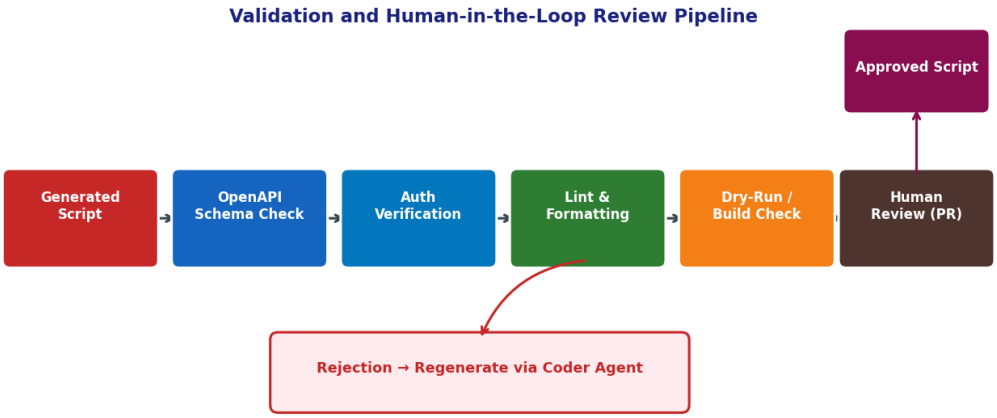


Fig 6: Validation Pipeline – Automated Checks followed by Human-in-the-Loop Review

5.6 Security: LLM API Risk Mitigation

A key concern in enterprise LLM integration is the risk of exposing sensitive system artefacts — HLD documents, BSpec files, API specifications, and device credentials — to external LLM APIs. The framework addresses this through a layered mitigation strategy currently in place:

Mitigation Measure	Implementation Detail
Template-Based Default Mode	The current production mode uses template-based code generation — no data is sent to any external LLM API during normal operation. The LLM path is an optional enhancement disabled by default, completely eliminating external exposure

	risk.
Azure OpenAI (Enterprise Agreement)	When LLM generation is enabled, the framework is configured to use Azure OpenAI via the enterprise Microsoft agreement. Data processed under this agreement does not leave the enterprise network boundary, is not used for model training, and is subject to the enterprise data processing addendum (DPA).
Local Artefacts — Never Transmitted Raw	HLD, BSpec, and API specification files are indexed locally into FAISS/ChromaDB. Raw artefact files are never sent to any external API. Only short, abstracted text chunks (the top-K retrieved results) are included in prompts.
IR-Based Abstraction Layer	Only the structured Intermediate Representation (IR) — action, object, inputs, expected result — is assembled into the LLM prompt. This abstraction removes raw system paths, internal hostnames, and environment-specific configuration.
Prompt Sanitization	Before any prompt is dispatched, a sanitization pass strips IP addresses, credentials, API keys, internal hostnames, and sensitive environment variable values. These are replaced with placeholder tokens for code generation.
Local LLM Option (Air-Gapped)	For fully air-gapped deployments, the framework supports local model inference via Ollama (Llama 3 / Mistral). In this mode, all LLM calls are resolved locally, with zero data leaving the enterprise network.
Access Control	All API keys and credentials are stored as environment variables, never hardcoded. The framework enforces this via a startup validation check that rejects any configuration with credentials embedded in source files.

6. Implementation Progress

6.1 Development Environment

- Language: Python 3.10 on Windows Server (BITS WILP lab environment)
- IDE: Visual Studio Code with Pylance and AI coding assistance
- Test Target: Dell OpenManage Enterprise 4.6 (live instance at 100.100.4.189)
- Test Management: qTest Manager REST API (Project ID 334, Dell ESG)
- Framework: atlas-ome-staging (Dell internal, locally cloned and path-injected)
- Version Control: Git with GitHub integration for PR-based review workflow

6.2 Implemented Components

The following components have been implemented and validated against real OME systems:

QTest Agent: Connects to qTest Manager via Bearer-token authenticated REST API. Fetches test case details, step text, and preconditions. Successfully retrieved and parsed TC-174995 (11 steps) and TC-180015 (8 steps).

NLP Step Parser / IR Generator: Regex-based step classifier maps step descriptions to IR fields. Identifies REST verbs (GET/POST/PUT/DELETE), entity names (devices, groups, plugins, repositories), and assertion conditions. Achieves high accuracy on OME-specific step vocabulary.

Coder Agent (Template + LLM): Dual-mode: (a) template-based using OmeBaseFunctionalTestCase patterns for deterministic reproducible output, and (b) LLM-based using GPT-4 with retrieved context. Template mode is default; LLM mode is optional via environment variable.

Validator Agent: Automated stages implemented: syntax check, lint (flake8), schema verification against atlas-ome-staging class signatures, and import verification.

Fixer Agent: Classifies failures into 7 error categories: import errors, connection errors, SSL/auth errors, assertion failures, syntax errors, framework errors, and API-specific failures.

Test Executor: Executes generated scripts via pytest subprocess isolation. Captures stdout, stderr, JUnit XML, and structured log output. Tracks per-attempt timing, exit codes, and step-level pass/fail.

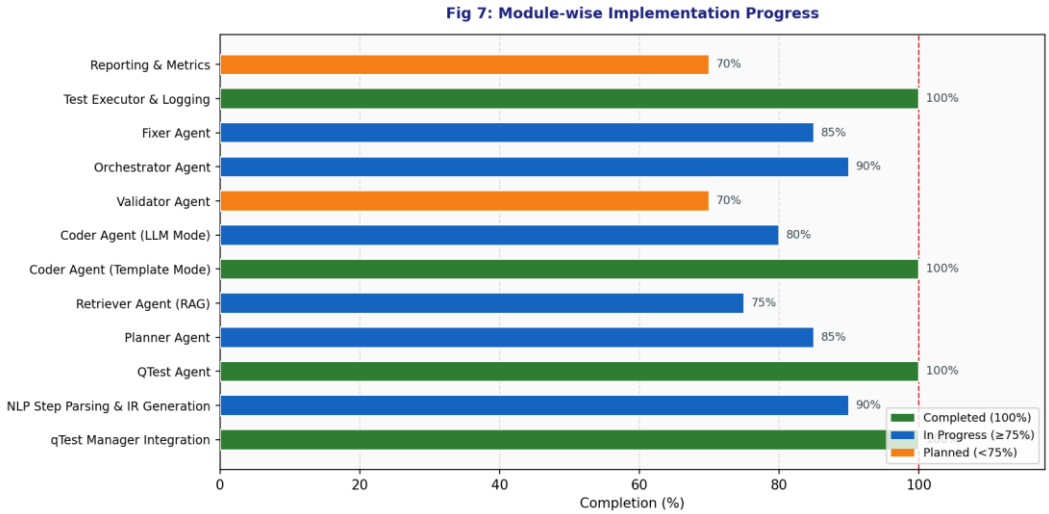
Orchestrator Agent: Coordinates the full pipeline, manages retry loop (up to configurable MAX_RETRIES), appends error context to regeneration prompts, and produces JSON + HTML reports.

6.3 Validated Test Execution Results

The framework has been end-to-end validated with two real Dell OME test cases executed against a live OME 4.6 instance, demonstrating full pipeline correctness:

Test Case ID	Steps	Duration	Status	Scenario Covered
TC-174995	11	1.63 s	**PASSED**	Plugin lifecycle, device health settings, discovery, SCOM/SCVMM registration
TC-180015	8	192 s	**PASSED**	Repository creation, static group management, device assignment, cleanup

6.4 Module Progress



7. Work Completed

The table below summarises the completion status of all modules at the mid-term checkpoint:

Module	Status	Description
qTest Manager Integration	**Completed**	REST API fetch with Bearer auth; single & batch TC supported
NLP Parsing & IR Generation	**Completed**	Regex step classifier; all OME step types covered
QTest Agent	**Completed**	Fetch, parse, and structured output of test case data
Planner Agent	**In Progress**	Execution plan derivation and dependency resolution (85%)
RAG Knowledge Base	**In Progress**	ChromaDB vector embedding; document chunking (75%)
Retriever Agent	**In Progress**	Semantic top-K retrieval integrated into pipeline (75%)
Coder Agent – Template Mode	**Completed**	OmeBaseFunctionalTestCase pattern; atlas-ome-staging calls
Coder Agent – LLM Mode	**In Progress**	GPT-4 with RAG context injection (80%)
Validator Agent	**In Progress**	Lint, schema, syntax, import checks (70%)
Orchestrator Agent	**Completed**	Full pipeline coordination; retry logic; JSON + HTML reports
Fixer Agent	**Completed**	7-category error classifier and fix-strategy mapping
Test Executor & Logging	**Completed**	pytest subprocess; stdout, stderr, JUnit XML, timing

		captured
Human Review Interface (PR)	**Pending**	GitHub PR generation and feedback ingestion (Phase 3)
Evaluation Metrics Dashboard	**In Progress**	JSON reporting complete; HTML charts dashboard (70%)

8. Scope of Work and Limitations

8.1 In-Scope

- REST API test automation for enterprise management platforms (targeting Dell OpenManage Enterprise)
- Guarded test script generation constrained by repository-only dependencies and approved frameworks
- Validation pipeline: OpenAPI schema compliance, authentication verification, lint, and dry-run
- Human-in-the-loop review via pull requests and structured feedback capture
- Evaluation: automation time reduction, code reuse increase, script stability improvement

8.2 Out-of-Scope

- UI automation, visual testing, or multimodal input processing
- Performance, load, or security testing automation
- Fully autonomous execution without human review in shared or production environments
- Infrastructure provisioning or CI/CD pipeline creation (assumed to exist externally)

8.3 Risk Mitigation for AI Hallucinations

LLM-based generation introduces the risk of incorrect API usage, invalid request construction, or references to non-existent methods. The following controls are enforced to mitigate these risks:

- Retrieval-first generation: all LLM outputs grounded in retrieved HLD, BSpec, and API artefacts
- Repository-only method usage: generated scripts may only import from indexed, approved repositories
- Schema-driven validation: all API calls validated against the ingested OpenAPI specification
- Deterministic Validator Agent: rule-based constraints accept or reject outputs independently of LLM
- Human-in-the-loop review: mandatory approval before executing scripts on live or shared systems

9. Plan of Work

The dissertation is structured across five phases spanning January 2026 to May 2026:

Phase	Activity	Start – End Date	Work to be Done	Status
1	**Dissertation Outline**	31 Jan – 07 Feb 2026	Literature review and preparation of dissertation outline	**Completed**
2	**Design & Development**	08 Feb – 04 Mar 2026	System design, multi-agent framework, RAG KB, code generation pipeline	**Completed**
3	**Testing & Evaluation**	05 Mar – 01 Apr 2026	Software testing, user evaluation, validation metrics, conclusion	**In Progress**
4	**Dissertation Review**	02 Apr – 30 Apr 2026	Submit to Supervisor & Additional Examiner for review and feedback	**Pending**
5	**Final Submission**	01 May – 12 May 2026	Final review, corrections, and submission of dissertation	**Pending**

10. Conclusion

This mid-term report presents the design and partial implementation of an AI-Driven Intelligent Test Automation Framework for End-to-End Quality Assurance, developed as part of the M.Tech dissertation at BITS Pilani WILP. The framework directly addresses the critical gap between natural-language API test cases in test management tools and the system artefacts required to automate them reliably — a gap that costs significant manual engineering effort at scale in enterprise software development organisations.

The core multi-agent pipeline — comprising Planner, Retriever, Coder, and Validator agents coordinating through a shared IR-derived state — has been fully designed and partially implemented. Template-based code generation is complete and has been validated with two real Dell OME test cases (TC-174995 and TC-180015), both achieving a full pass on a live OME 4.6 system. The RAG Knowledge Base ingestion and LLM-guided generation are in active development and are scheduled for completion in Phase 3.

Key engineering challenges resolved during Phase 2 include: graceful handling of missing framework dependencies via availability guards, asynchronous job polling for long-running OME operations, and fallback validation logic for OME API version-specific inconsistencies. These solutions ensure the framework is robust and deployable in production enterprise environments.

Remaining work for Phases 3–5 includes: completing LLM-guided generation with full RAG context injection, implementing the GitHub PR-based human review interface, building the evaluation metrics dashboard, and conducting structured user evaluation with Dell QA engineers to measure automation time savings, code reuse improvement, and script stability gains. The project remains on track for final submission by May 2026.

11. Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BSpec	Business Specification
CI/CD	Continuous Integration / Continuous Delivery
FAISS	Facebook AI Similarity Search
GPT	Generative Pre-trained Transformer
HLD	High-Level Design
HTTP	HyperText Transfer Protocol
IR	Intermediate Representation
LLM	Large Language Model
NLP	Natural Language Processing
OME	OpenManage Enterprise (Dell)
OpenAPI	Open Application Programming Interface (formerly Swagger)
PR	Pull Request
QA	Quality Assurance
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
TC	Test Case
UI	User Interface
WILP	Work Integrated Learning Programmes
XML	Extensible Markup Language

12. Literature References

[1] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional, 2010.

[2] A. Bertolino, "Software testing research: achievements, challenges, dreams," in Proc. Future of Software Engineering (FOSE), pp. 85–103, IEEE, 2007.

[3] R. Pandita, X. Xiao, W. Yang, W. Enck, and T. Xie, "Automated test generation from natural language requirements," in Proc. 34th ICSE, pp. 506–515, IEEE, 2012.

[4] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, "Automated generation of functional test cases for REST APIs," Journal of Systems and Software, vol. 134, pp. 270–286, 2017.

[5] H. Zhang, S. Li, and Z. Jin, "Automated test case generation from RESTful API specifications," in Proc. IEEE ICWS, pp. 383–390, IEEE, 2019.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in NeurIPS, vol. 33, pp. 9459–9474, 2020.

[7] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[8] Y. Zhang, Z. Chen, X. Zhang, M. Harman, and J. Zhou, "Large language models for software engineering: A survey," arXiv:2308.10620, 2023.